



PDF Download
3664476.3664498.pdf
16 February 2026
Total Citations: 3
Total Downloads: 598

Latest updates: <https://dl.acm.org/doi/10.1145/3664476.3664498>

RESEARCH-ARTICLE

Graph-Based Spectral Analysis for Detecting Cyber Attacks

MAJED JABER, EPITA, Le Kremlin-Bicetre, Ile-de-France, France

NICOLAS BOUTRY, EPITA, Le Kremlin-Bicetre, Ile-de-France, France

PIERRE PARREND, EPITA, Le Kremlin-Bicetre, Ile-de-France, France

Open Access Support provided by:

EPITA

Published: 30 July 2024

[Citation in BibTeX format](#)

ARES 2024: The 19th International
Conference on Availability, Reliability
and Security
July 30 - August 2, 2024
Vienna, Austria

Graph-Based Spectral Analysis for Detecting Cyber Attacks

Majed JABER

ICube, UMR 7357, Université de
Strasbourg, CNRS
Strasbourg, France
Laboratoire de Recherche de l'EPITA,
14-16 Rue Voltaire
Le Kremlin-Bicêtre, France
majed.jaber@epita.fr

Nicolas BOUTRY

Laboratoire de Recherche de l'EPITA,
14-16 Rue Voltaire
Le Kremlin-Bicêtre, France
nicolas.boutry@lrde.epita.fr

Pierre PARREND

ICube, UMR 7357, Université de
Strasbourg, CNRS
Strasbourg, France
Laboratoire de Recherche de l'EPITA,
14-16 Rue Voltaire
Le Kremlin-Bicêtre, France
pierre.parrend@epita.fr

ABSTRACT

Spectral graph theory delves into graph properties through their spectral signatures. The eigenvalues of a graph's Laplacian matrix are crucial for grasping its connectivity and overall structural topology. This research capitalizes on the inherent link between graph topology and spectral characteristics to enhance spectral graph analysis applications. In particular, such connectivity information is key to detect low signals that betray the occurrence of cyberattacks. This paper introduces SpectraTW, a novel spectral graph analysis methodology tailored for monitoring anomalies in network traffic. SpectraTW relies on four spectral indicators, Connectedness, Flooding, Wiriness, and Asymmetry, derived from network attributes and topological variations, that are defined and evaluated. This method interprets networks as evolving graphs, leveraging the Laplacian matrix's spectral insights to detect shifts in network structure over time. The significance of spectral analysis becomes especially pronounced in the medical IoT domains, where the complex web of devices and the critical nature of healthcare data amplify the need for advanced security measures. Spectral analysis's ability to swiftly pinpoint irregularities and shift in network traffic aligns well with the medical IoT's requirements for prompt attack detection.

CCS CONCEPTS

• Security and privacy → Intrusion/anomaly detection and malware mitigation; Intrusion detection systems.

KEYWORDS

Cybersecurity, Graphs, Spectral Graph Analysis, Spectrum, Anomaly Detection, Laplacian Matrix

ACM Reference Format:

Majed JABER, Nicolas BOUTRY, and Pierre PARREND. 2024. Graph-Based Spectral Analysis for Detecting Cyber Attacks. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024)*, July 30–August 02, 2024, Vienna, Austria. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3664476.3664498>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ARES 2024, July 30–August 02, 2024, Vienna, Austria

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1718-5/24/07

<https://doi.org/10.1145/3664476.3664498>

1 INTRODUCTION

The IoT [35] is revolutionizing the landscape of healthcare through the integration of medical devices and systems with internet capabilities [22, 23]. In the realm of medical IoT, devices such as wearable health monitors, implantable sensors, and diagnostic machines are interconnected to facilitate remote patient monitoring, real-time data collection, and personalized healthcare delivery [52].

However, the adoption of IoT in the medical domain also introduces unique security challenges [50] due to the sensitive nature of healthcare data and the critical implications of potential cyber threats. In particular, the use of lightweight, low-power IoT devices in medical settings [26] has outpaced the development of robust security protocols, leaving medical IoT networks vulnerable to various cyber attacks. We focus in particular on reconnaissance and theft attacks, which can compromise patient privacy, disrupt healthcare services, and even endanger patient lives. Therefore, there is a pressing need for sophisticated mechanisms for threat detection and response tailored to the requirements of medical IoT environments.

In response to these challenges, this paper proposes a spectral time windowing approach, termed *SpectraTW*, for attack detection specifically within medical IoT network traffic. We propose four metrics to evaluate connectivity: *Connectedness*, *Flooding*, *Wiriness*, and *Asymmetry*. We evaluate these metrics using four methods of increasing expressivity: Classification on original data log (COD), Classification on Time series (CTS), Classification on Time window (CTW), as well as *SpectraTW* that we propose in this study. By modeling network interactions as a graph, *SpectraTW* enables the identification of anomalous behaviors that may indicate security breaches or potential threats to patient safety. Preliminary results from this research demonstrate the effectiveness of graph spectral analysis in enhancing the performance of machine learning (ML) algorithms [8, 36, 45, 47], such as XGBoost [15], in classifying and predicting network anomalies within medical IoT datasets. This evaluation provides initial insights into the applicability of *SpectraTW* in real-world medical IoT scenarios, laying the foundation for further development in this critical area of research.

The structure of the paper is as follows: Section 2 reviews the current literature on graph extraction from network data. It details how eigenvalues play a role in this conversion and discusses the initial research on star graphs. Section 3 provides a detailed examination of the datasets used, specifically the Botnet and TonIoT datasets, including the specifics of the attacks they contain. Section 4 defines

the four proposed metrics. Section 5 discusses the four methodologies we explore: COD, CTS, CTW, and *SpectraTW*. Section 5.3 details the experiments conducted, applying these methodologies to the datasets with particular constraints. Section 5.4 discusses the results and evaluations of these experiments. Section 5.5 highlights the key features of our study and elucidates the effectiveness of the metrics used. Section 6 focus on attack behaviors and models. The paper concludes with Section 7, summarizing the findings and contributions of this work.

2 STATE-OF-THE-ART

In this section we talk about the existing approaches in the literature, the contribution of ML to the cybersecurity domain for anomaly detection through statistical, graph-based, and deep learning strategies. It also highlights the role of spectral graph analysis in identifying anomalies, explaining how network traffic can be represented as graphs, from foundational concepts in graph theory to spectrum of graphs.

2.1 Machine Learning for anomaly detection

Anomaly detection [1] main approaches are *statistical ones* [2] and ML-based ones [6]. Among the most recent statistical approaches, we can refer to a real-time network anomaly-detector called *ReTiNA* [42] and an unsupervised three-stage framework for detecting anomalous network behavior on-the-fly. We can also cite [28] presenting a scalable, principled, probability-based technique for detecting outlying connectivity behaviors. Traditional systems use elementary statistics techniques and are often inaccurate [27]. For this reason, a ML-based approach called Cybersecurity autonomous machine learning platform for anomaly detection (CAMLPA) has been proposed [27]. This streamlined, holistic approach begins with retrieving a multitude of different sources of cybersecurity data like log files parsed as JSON and feeded as network traffic data in real time using Elasticsearch, then runs several ML algorithms. The calculated anomalies are assigned an outlier score which serves as an indicator for whether an alert should be sent to the system administrator that there are potential anomalies in the network. ML-based techniques are algorithms like Logistic Regressions, Decision Trees, Naive Bayes techniques, Support Vector Machines, k -Nearest Neighbors, and Random Forests for the supervised ones and k -Means and variants for the unsupervised ones.

In network security, there are not much labeled data to train efficient classifiers [10] including data skewness, where most samples are normal, making rare threats hard to detect; label noise, where inaccuracies in labeling can mislead the training process; and the evolving nature of cyber threats, which means datasets quickly become outdated, reducing the effectiveness against new attacks. Furthermore, existing labeled data are often specific to the context and thus may not be useful for other applications. Hopefully, graph-based machine learning techniques have the potential to make significant impact in the next-generation cybersecurity systems. Walk-based sampling is a technique whereby graph structured data is sampled via walks through the graph (please refer to [25, 44]). In the simplest case, random walks are performed, which converts the unstructured graph data into structured sequences of nodes and edges which can then be processed by traditional ML techniques.

In [11], the authors detect lateral intrusions thanks to unsupervised learning of graphs. In [13], the authors propose StrGNN, an end-to-end structural temporal GNN model for detecting anomalous edges in dynamic graphs.

Deep learning [38, 55] has received widespread concerns in the graph data field. Kipf *et al.* [29] proposed a generalisation method of *Graph Convolutional Networks* (GCN's), currently the best choice for graph data learning tasks. HyperGCN [54] generalized the simple graph convolution operation of GCN to the hypergraph domain by using hypergraph Laplacian to capture more complex or beyond pairwise relationships between nodes. Graph attention networks [51] encode the hidden representations of each node in the graph by introducing the self-attention mechanism to attend over its neighbors. Liu *et al.* [33] propose a weakly supervised multi-label image classification framework based on GCN with learning the semantic label co-occurrence in an image.

2.2 Spectral Graph Analysis for attack detection

Several approaches utilizing spectral analysis have been developed for cybersecurity applications. In [17], the authors detect changes in the periodicity of Transmission Control Protocol (TCP) packet transport associated with round-trip times: using power spectral density (PSD), they can defend against denial of service (DoS) attacks. Another approach [53] employs graph theory, diffusion, and spectral methods to analyze forensic evidence. This method demonstrates the potential of graph-spectral and kernel-based methods in extracting structural characteristics of the evidence graph. A significant result of leveraging such graph-based methods is their ability to accurately identify and profile distinct attack scenarios, thereby facilitating the creation of detailed attack case profiles through the topological features that enhance forensic analysis and threat intelligence. By modeling the propagation of suspicion in the attack scene using heat diffusion, this approach extracts attack scenarios and creates attack case profiles. A method [14] called MC-GPCA has been introduced to reduce the complexity of a single graph by breaking it down into several centrality features and reducing their correlation. In the same paper, another method called MC-GDL performs dictionary learning on a group of graphs using multiple centrality features. Both methods utilize spectral decompositions using singular value decomposition (SVD). In [56], a fraud detection framework is developed to identify various cybersecurity attacks by analyzing the eigenvalues and eigenvectors of the adjacency matrix. This approach studies how to identify attackers by characterizing their distributions in the spectral space. The scheme proposed in [7] supports the discovery of static graphs patterns, model dynamic graphs, cluster evolving graphs, and the detection of anomalies in dynamic graphs. Last, in [16], the authors merge discrete Fourier transform (DFT) with a hypothesis testing framework to address shrew attacks.

2.3 Graph Representation: Foundations and Applications in Cybersecurity

This subsection explores graph theory's role in cybersecurity, using graphs to model networks for threat identification. It details how graphical representations analyzes vulnerabilities and emphasizes the criticality of matrices like adjacency and Laplacian, in evaluating

network strength and detecting anomalies, underscoring the value of graph analytics in enhancing security.

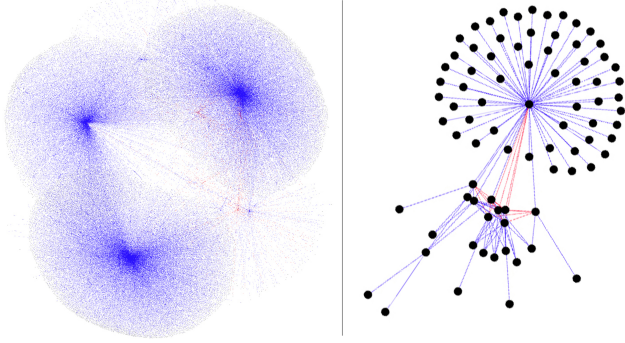


Figure 1: Graph representations shows the TonIot and Botnet datasets respectively from the left to the right.

2.3.1 Defining the Network, Core Concepts and Structures: A network in information technology is described as an assembly of interconnected devices or *nodes*—including computers, servers, switches, and routers—that are designed for data exchange and resource sharing. These networks are characterized by *interconnectivity*, allowing devices to connect via wired or wireless methods; *Data Exchange*, facilitating the movement of files and messages; *Shared Resources*, enabling the communal use of assets like printers and internet connections; and adherence to *Protocols*, which are predetermined rules that ensure devices communicate effectively [24, 34, 48].

2.3.2 Networks in Graph Theory, A Comparative Analysis: Networks are visualized as graphs with devices and their interactions represented as nodes and edges, respectively, as illustrated in Fig. 1. This visualization aids in understanding the complex dynamics of networks, particularly for cybersecurity purposes. Nodes denote various network devices, potentially including both secure and compromised units, while edges symbolize the communications between them, with attributes such as data rates and protocol types. This graphical model leverages graph analytics to identify anomalies and patterns indicative of cybersecurity threats, such as botnet activities, by examining the deviations in the communication patterns among the networked devices.

2.3.3 Fundamentals of Graph Theory, Exploring Eigenvalues. When a network is represented by a graph, we can extract topological properties thanks to spectral graph analysis [18] using their Laplacian spectrum. Several features follow: the number of eigenvalues equal to zero (the number of connected components of the graph), the greatest eigenvalue (bipartiteness [5]), etc. Robustness [21] to edge rewiring of this spectrum has been shown in the sense that *a small modification of the graph weights implies a small change in the graph spectrum*.

Structural Matrices in Graph Theory: Consider a graph denoted as G with a number n of nodes. Its structure is represented by three key matrices: First, its *Adjacency Matrix* A , an $n \times n$ matrix, where

the element A_{ij} is 1 if a connection exists between nodes i and j , and 0 otherwise. Next, the *Degree Matrix* D of G is a diagonal $n \times n$ matrix where the i^{th} diagonal element D_{ii} is set at $\sum_{j=1}^n A_{ij}$, the off-diagonal entries being set at 0. Lastly, the *Laplacian Matrix* L is defined as $L = D - A$ [37]. This matrix is critical in the domain of spectral graph theory, encapsulating the graph’s spectral attributes and its structural coherence.

Eigenvalues in Graph Theory: Analyzing the Laplacian Matrix Spectrum: Let Λ be the spectrum of the Laplacian matrix L , $\mathcal{Z}$ is equal to the number of eigenvalues equal to 0 in Λ . We recall that, in linear algebra, any eigenvalue λ of a square matrix A , satisfies the relation $Av = \lambda v$ [12], with v the eigenvector associated to the eigenvalue λ . This concept of eigenvalues and eigenvectors is pivotal in various mathematical and scientific fields, especially in examining the spectrum of the Laplacian matrix within graph theory. The Laplacian matrix’s spectrum is invaluable as it uncovers critical structural properties of graphs, maintaining its relevance even when the graph’s topology is modified [12], thereby emphasizing its utility in exploring and understanding the convolutions of medical networks.

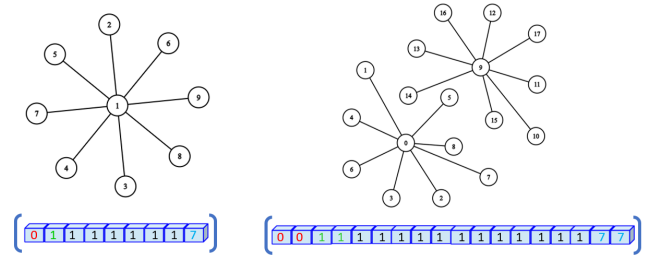


Figure 2: Two visual representations illustrate star graph topologies: the first on the left showcases a single connected component, while the second on the right displays two connected components, each accompanied by their spectrum.

Interesting Eigenvalues in Graph Topologies: The focus was in-depth examination of various graph structures, particularly emphasizing the eigenvalues of the Laplacian matrix. Star graphs, two examples are shown in Fig. 2, commonly found in the studied datasets, are the primary subject of the initial experimental phase. These experiments aim to replicate both standard and irregular behaviors within star graph structures. Key to this analysis is the observation of spectral shifts within the graphs, indicative of potential security risks. This observation leads to the development of four spectral metrics, grounded in the unique properties of certain eigenvalues as we will discuss in Section 4, enabling the detection and analysis of unusual network patterns. In terms of spectral analysis, the zero eigenvalues of a graph’s Laplacian matrix are instrumental in determining its connected components [12]. A number of k zero eigenvalues is indicative of k isolated components within the graph; their cardinality is denoted $\mathcal{Z}$ (or more generally $\mathcal{Z}(t)$ when we study dynamic graphs). The algebraic connectivity, or the *Fiedler value* [20], of a graph is another critical measure, derived from the second smallest eigenvalue of the Laplacian matrix. It reveals the structural coherence and connection strength of the graph, with

higher values denoting greater integration. The Fiedler vector finds applications in graph partitioning and community detection. Moreover, the spectral radius, which is the largest eigenvalue of the Laplacian matrix, serves as a critical indicator of a graph's overall connectivity and its resilience to disruptions [12, 32, 57]. A higher spectral radius generally points to a more interconnected network, closely associated with the degrees of the nodes in the graph.

3 CYBERSECURITY CASE STUDIES

This section presents the Botnet and TonIoT datasets, key to identifying network attacks such as scanning and theft attacks and distinguishing them from normal traffic.

3.1 Data Exploration

Our study utilizes the Botnet [31] and TonIoT [39] medical IoT datasets, which contain simulated network traffic logs include common features like timestamps, packets, size in bytes, source and destination ports, protocols used, and IP addresses [30]. Our approach focuses on these shared aspects to enhance our model's applicability to a variety of network environments. The datasets, typically in PCAP format, are crucial for reconstructing network scenarios such as DDoS flooding attack.

3.1.1 Botnet-IoT Dataset. The Bot-IoT dataset, developed for IoT environments, is generated through a testbed network platforms with both regular and attacking VMs, IoT services simulated via Node-red (e.g., weather stations), and the Argus tool for feature extraction [9]. Normal traffic is replicated using smart-home devices like smart refrigerators and thermostats, communicating via MQTT with AWS, while malicious activities are executed by Kali Linux VMs, including attacks like probing, port scanning, DoS, and data theft, using tools such as Nmap and Metasploit [30]. This diverse dataset, therefore, emulates a real-world IoT network, aiding in the development of robust cybersecurity solutions.

3.1.2 Ton-IoT Dataset. The ToN IoT dataset, aimed at evaluating AI-based cybersecurity applications generated from a sophisticated testbed at UNSW Canberra Cyber's IoT Lab, it encompasses a broad range of data sources including telemetry from IoT/IIoT sensors, various operating systems, and network traffic [9]. The testbed is structured into three layers: edge (with physical IoT/IIoT devices and operating systems), fog (hosting VMs and services, including client and offensive systems), and cloud. This setup facilitates the recording of diverse cyberattacks such as DoS, DDoS, ransomware, and MITM, using tools like Nessus and Metasploit. The dataset's features include detailed connection, DNS, SSL, HTTP, and violation activities, capturing packet captures, Bro logs, sensor data, and OS logs. This makes the ToN IoT dataset exceptionally comprehensive in terms of feature variety and IoT device diversity [3, 40, 41].

3.2 Attack Profiling: Analysis and Distributions

Fig. 3 presents the distribution of various attack types in the Botnet and TonIoT datasets. In the Botnet dataset, Distributed Denial of Service (DDoS) attacks are most frequent, with 576,894 instances, other less frequent attacks include Reconnaissance and Theft. Notably, UDP flood attacks, Service Scans, and Operating System Fingerprinting are significant subcategories.

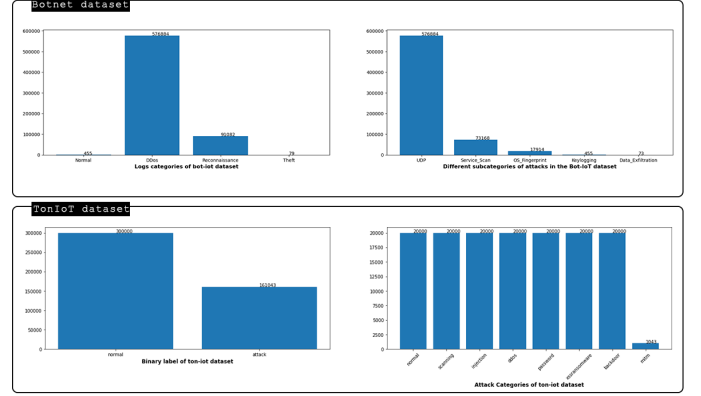


Figure 3: Distribution of normal traffic, attacks and attack categories of Botnet and TonIoT data-sets

Conversely, the TonIoT dataset categorizes events into "normal" and "attack" types, with 300,000 normal and 161,043 attack instances. The attack category includes Scanning, Injection, DoS, Password, Exploitation, Backdoor, and MitM attacks, each averaging around 20,000 instances, except for MitM with 1,043 occurrences, demonstrating a diverse range of cyber attack techniques.

4 INTRODUCING SPECTRAL METRICS

In our research, we propose to leverage the expressiveness of the Laplacian matrix spectrum, focusing on the eigenvalues' significance across various graph topologies which act as weak signals betraying specific cyberattacks. Through our analysis and dataset reviews, we identify star graphs as prevalent structures, guiding our initial experiments. These experiments are based on normal and abnormal network behaviors within star graph configurations, closely observing spectral changes to detect unusual activities. We develop four spectral metrics based on distinctive eigenvalues, that prove to be efficient for detecting and analyzing anomalous network patterns. The four metrics are formalized as follows:

In this context, $\Lambda^{(t)}$ is the spectrum of L at time t , and, for $i \in [1, n]$, $\lambda_i^{(t)}$ is the i^{th} smallest value in $\Lambda^{(t)}$.

4.1 Connectedness

The first metric we introduce defines a measure that assesses the degree of inter-connectedness within the network. This metric is named *Connectedness* and is represented as $\mu_1(t)$. Its formulation is described as follows:

$$\mu_1(t) = \frac{\exp \frac{1}{Z(t)}}{\exp(1)}$$

In this equation, $Z(t)$ is the number of zero values in the network's spectrum. The aim of this metric is to identify interconnections within the network. When μ_1 approaches 1, it suggests a potential intrusion, since it implies the formation of new links between connected components. It's worth noting that as $Z(t)$ tends towards $+\infty$, μ_1 converges to $\exp^{-1} \approx 0.367$. Conversely, when $Z(t)$ approaches 1, μ_1 tends to 1.

4.2 Flooding

Now, let's delve into our second metric, where we aim to take into account the whole connectivity of the network graph, and that can be studied by taking into consideration the algebraic connectivity of each connected component. Let N be the number of network devices (e.g. switches and servers). This metric, denoted as $\mu_2(t)$, is defined as follows:

$$\mu_2(t) = \left(\frac{1}{N} \sum_{i=Z(t)+1}^{Z(t)+N} \lambda_i^{(t)} \right) - 1$$

μ_2 takes into consideration both the presence of connections between nodes or devices and their associated weights like number of packets or bytes depending on feature studied over the time-window. When there is a high number of connections with significant connecting weights, this metric, referred to as *Flooding* then increases accordingly.

4.3 Wiriness

For the third metric, known as *Wiriness* and represented as $\mu_3(t)$, it approaches the consideration of interconnections and edge weights in a distinct manner. The definition is as follows:

$$\mu_3(t) = \frac{1}{N} \sum_{i=n-N+1}^n \lambda_i^{(t)}$$

μ_3 is sensitive to the existence of connections and their associated weights. It exhibits a tendency to rise when connections are established, with the rate of increase being contingent on variables like packet sizes (weights) and the quantity of connections. Typically, it experiences an upsurge when transmitted packets are larger, and there is a high volume of requests.

4.4 Asymmetry

The fourth and last metric, denoted as $\mu_4(t)$, is named *Asymmetry*. Its definition is as follows:

$$\mu_4(t) = \text{Cardinality} \left(\left\{ i \in [2, n] ; \lambda_i^{(t)} - \lambda_{i-1}^{(t)} > \varepsilon \right\} \right)$$

μ_4 serves as a measure of the variations in $\Lambda^{(t)}$ as the index i in $\Lambda^{(t)}$ goes from 2 to n . When the network exhibits numerous identical patterns, this metric tends to be low. However, in the presence of connections, the curve depicted by $i \rightarrow \lambda_i^{(t)}$ is smoother and the asymmetry metric increases, indicating a potential security threat. The inclusion of $\varepsilon = 10^{-12}$ in the formula helps mitigate approximation errors.

Together, these metrics form a robust framework for evaluating the dynamics of a network and discerning possible cybersecurity risks. They offer a holistic approach to assessing various aspects of network activity, enabling a more thorough analysis of potential threats to network security. This multifaceted approach enhances our understanding of network behavior and strengthens our ability to safeguard against evolving security challenges.

5 METHODOLOGIES AND TECHNIQUES

In this section, we go through a series of methodologies aimed at improving network security analysis. Starting from basic dataset examination to sophisticated spectral-time-window classification, each approach builds upon the last to offer a gradating understanding of network behavior and threat detection. Through these methods, we aim to set a foundation for identifying and analyzing anomalies within network traffic, emphasizing the exclusion of time and sequence data to mitigate bias and enhance detection accuracy.

Table 1: Feature selection for each approach

	COD	CTS	CTW	SpectraTW
Number of packets	x	x	x	x
Size of byte	x	x	x	x
Packets rate	x	x	x	x
Duration	x	x		
Protocol	x	x		
State	x	x		
Mean, Sum, Min, Max	x	x		
Connectedness				x
Flooding				x
Wiriness				x
Asymmetry				x

5.1 Methodological Approaches

Next, we will present the four methods we have implemented, along with their respective methodologies, beginning with COD, then progressing to CTS and CTW, and ending up with *SpectraTW*.

5.1.1 Classification on original data-logs (COD). In this approach, that serves as a baseline for subsequent evaluations, we work with datasets without any preprocessing steps. We include all dataset features except those related to date and time. This exclusion is because using time-related features during the learning phase could lead to biased results, as most attacks tend to occur at the same timestamp. As well as features related to the sequence of packet arrivals, are disregarded. The primary objective of this approach is to establish a baseline for comparison with other methods [19, 46]. Our approach involves three main steps: First, we consider the entire dataset without any preprocessing phase. Second, we include all dataset features, excluding sequence and time-related ones. Finally, later on, we will apply various classification models to the dataset and evaluate them using default ML metrics.

5.1.2 Classification on Time-series (CTS). This approach represents a progression from the previous one, as it also utilizes all the dataset's features while excluding time and sequence attributes, as shown in Algorithm 1, this step is represented by Line 1 using the method *lighten*. The motivation behind this approach arises from the observation that the data logs in the dataset are imbalanced between attack and normal traffic. Furthermore, leveraging time series analysis offers significant advantages when dealing with dynamic graphs evolving over time [43]. Each unique timestamp in the dataset represents an evolution of the graph.

To implement this approach, we follow the process shown in Fig 4 by aggregating data based on specific constraints and grouping

Table 2: Aggregation methods for dataset features

Feature	Aggregation method
Total packets	Sum
Source packets	Sum
Destination packets	Sum
Total bytes	Sum
Source bytes	Sum
Destination bytes	Sum
Sum	Mean
Label	First
Total rate	Mean
Source rate	Mean
Destination rate	Mean
Duration	Mean
Mean	Mean
Sum	Mean
Max	Mean
Min	Mean

	stime	saddr	daddr	pkts	attack	weight	
Merge	576923	1526344032	192.168.100.46	192.168.100.5	59452	0	1
	576917	1526344032	192.168.100.46	192.168.100.5	30157	0	1
	576916	1526344032	192.168.100.5	192.168.100.3	29726	0	1
	576921	1526344033	192.168.100.7	13.55.154.75	3018	1	1
	576884	1526344121	192.168.100.1	192.168.100.3	4	0	1

	stime	saddr	daddr	pkts	attack	requests
0	1526344032	192.168.100.46	192.168.100.5	$\sum pkts = 89,609$	0	$\sum weight = 2$
		192.168.100.3	13.55.154.73	$\sum pkts = 29726$	0	1
1	1526344033	192.168.100.7	13.55.154.75	$\sum pkts = 3018$	1	1
2	1526344121	192.168.100.1	192.168.100.3	$\sum pkts = 4$	0	1

Figure 4: CTS over dataset, Botnet traffic network sample**Algorithm 1 CTS****Require:** $ds \leftarrow Dataset, t \leftarrow 1sec$ **Ensure:** $CTS(ds)$

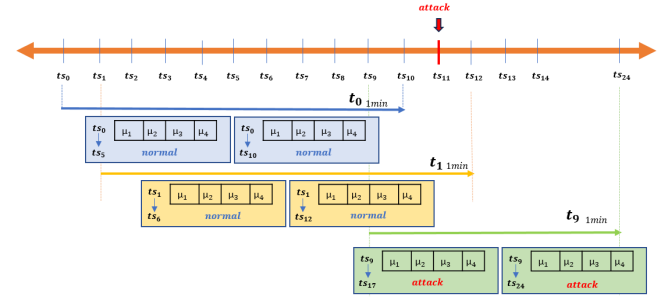
```

1:  $COD(ds) \leftarrow lighten(ds)$ 
2:  $CTS(ds) \leftarrow []$ 
3: for all  $ts \in COD(ds).getUnique()$  do
4:    $ts\_Logs \leftarrow COD(ds).getLogs(ts)$ 
5:    $grp\_Logs \leftarrow ts\_Logs.groupBy(srcIP, dstIP, label)$ 
6:   for all  $g \in grp\_Logs$  do
7:     if  $g.count() > 0$  then
8:        $CTS(ds).add(Aggregate(g))$ 
9:     end if
10:  end for
11: end for

```

criteria. Initially, we define a time constraint t ($t \geq 1sec$) for each graph evolution. Then, iterating over the set of logs as shown in line 3 on specific time t represented by ts . After that, the grouping condition involves identifying traffic logs from the same source IP to the same destination IP with the same traffic type (e.g., normal), this is the core concept of this approach as shown in Algorithm 1 Line 5. If such traffic occurs within the same range t , it is aggregated Line 8. The aggregation process is feature-specific, with each feature having its own aggregation method. Consequently, through this

procedure, we transform the dataset into a time-series $CTS_{dataset}$ format with well-defined constraints on time and features.

**Figure 5: Times-window methodology over TS**

5.1.3 Classification on Time-window (CTW). In this approach, we advance our study by delving deeper into the representation of topological structure graphs within the context of timeseries data. To narrow down the time range where attacks occur, we propose the use of a time-windowing approach [4, 49]. This method revolves around the utilization of a sliding window that moves across the time-series data $CTS_{dataset}$, examining the quantitative and topological features within each sliding window. This ensures the adaptability of our methodology to change in the topological structure.

Specifically, the sliding window has certain constraints that dictate its operation within the time-series. These constraints include the time range of the sliding window tw , and the minimum number of data logs it must consider within each time window as MIN_{tw} (where, $MIN_{tw} \geq 2$) as shown in Algorithm 2 on Line 2. Let's delve into the process that occurs within each sliding window at time t_i .

As shown in Fig. 5, during each time window tw , the focus is exclusively on the extraction of quantitative features, deliberately excluding the topological structure. This aspect will be addressed in the upcoming approach, detailed in Section 5.1.4, along with our spectral metrics. The specific features extracted within tw encompass packet, byte, and rate features as written on Line 4, which are already a part of $CTS_{dataset}$, yet other features are not included. The exclusion of these additional features is strategic, aiming to facilitate a comparison with the spectral approach, and to highlight the significant potential of spectral metrics that rely on such features, without the need for further attributes for the moment.

It's worth noting that a sliding window may consider two or more timestamps (data rows) as it moves through the time-series. Our objective is to create a new dataset where each data row represents an aggregation of the data rows within the same sliding window. This aggregation follows the same methods outlined in Table 2 and is labeled based on the presence or absence of attacks within the sliding window. If an attack is present, the data row is labeled accordingly; otherwise, it is labeled as a normal data row. Once we have generated this new data-frames denoted as TW_{bot} and TW_{ton} , which counts according to the number of sliding windows within the timeseries for Botnet-IoT and Ton-IoT datasets respectively.

5.1.4 Spectral Timewindow (SpectraTW). In this method, we utilize the graph structures within each designated time-window This

Algorithm 2 CTW**Require:** $CTS(ds), tw, MIN_{tw}$ **Ensure:** $CTW(ds)$

```

1:  $TW(ds) \leftarrow []$ 
2:  $SW \leftarrow CTS(ds).getWindows(tw, MIN_{tw})$ 
3: for each  $w$  in  $SW$  do
4:    $F \leftarrow w.Features([pkts, bytes, rates])$ 
5:   if  $attack \in w$  then
6:      $l \leftarrow attack$ 
7:   else
8:      $l \leftarrow normal$ 
9:   end if
10:   $TW(ds).add(Aggregate(F, l))$ 
11: end for

```

approach seems similar to the previous approach CTS, with a significant distinction in the handling of the time-window.

Step 1. Data rows are converted into a graph, then processed in two separate sub-time-windows, Sub_{TW_1} and Sub_{TW_2} shown in Algorithm 3 on Line 3. Sub_{TW_1} is the graph constructed from TW_0 to $TW_{n/2}$, while Sub_{TW_2} covers the graph from TW_0 to TW_n where n is the total number of timestamps within the time-window. Dividing a time-window into two sub-windows aims to monitor any variations within the time-window, which would directly impact the metrics.

Step 2. After forming graphs over sub-time-windows, we extract the Laplacian matrix L of Sub_{TW_1} and Sub_{TW_2} (Line 11), resulting in L_1 and L_2 respectively (Line 12). Subsequently, we determine the spectra Λ_1 and Λ_2 for each Laplacian matrix (Line 13).

Finally, a set of four metrics, μ_1, μ_2, μ_3 , and μ_4 , are derived from each sub-time-window (Lines 14 and 15). After generating the new datasets, named $SpectraTW_{bot}$ and $SpectraTW_{ton}$, which reflect the number of sliding windows across the timeseries for both Botnet-IoT and Ton-IoT datasets, we advance to the application of various classification techniques.

5.2 Implementation

To execute this method, we utilized two Python notebooks, each following identical methodologies of *COS*, *CTS*, *CTW* and *SpectraTW* but applied to distinct datasets: one for the Botnet dataset and the other for the TonIoT dataset. In the interest of promoting reproducibility and push forward research within the field of cybersecurity, we have shared the source code for the metrics and their evaluation on GitHub¹. This repository contains required scripts for reproducing the experiments.

5.3 Experiments: Practical Applications

This section presents the applications and case studies conducted as part of our research, focusing on the Botnet-IoT and Ton-IoT datasets. We outline the methodologies applied to these datasets, starting with feature selection. The experiments progress through various stages, from the application of the *COD* method for initial

Algorithm 3 SpectraTW**Require:** $ds \leftarrow Dataset, tw, MIN_{tw}$ **Ensure:** $SpectraTW(ds)$

```

1:  $SW \leftarrow CTS(ds).getWindows(tw, MIN_{tw})$ 
2: for each  $w$  in  $SW$  do
3:    $Sub_{TW_1}, Sub_{TW_2} \leftarrow Split(Graph(w))$ 
4:    $F \leftarrow w.Features([pkts, bytes, rates])$ 
5:   if  $attack \in w$  then
6:      $l \leftarrow attack$ 
7:   else
8:      $l \leftarrow normal$ 
9:   end if
10:  for each  $i$  in  $[Sub_{TW_1}, Sub_{TW_2}]$  do
11:     $G(i) \leftarrow ConvertToGraph(i)$ 
12:     $L(i) \leftarrow Laplacian(G(i))$ 
13:     $\Lambda_i \leftarrow Spectrum(L(i))$ 
14:    Extract metrics  $\mu_1, \mu_2, \mu_3, \mu_4$  from  $\Lambda_i$ 
15:     $V_i \leftarrow [\mu_1, \mu_2, \mu_3, \mu_4]$ 
16:  end for
17:   $r \leftarrow row(Aggregate(F, V_{Sub_{TW_1}}, V_{Sub_{TW_2}}, l))$ 
18:   $SpectraTW(ds).add(r)$ 
19: end for

```

feature processing to the *CTS* and *CTW* methods for transforming the data into time series and aggregated time-window formats, respectively. Finally, the application of *SpectraTW* approach, which further analyzes the data within specific time windows using spectral graph analysis. Each subsection provides insights into the methodology applied, the rationale behind feature selection, and the impact of these choices on the analysis outcomes. Through these experiments, we aim to demonstrate the efficacy of our approaches in detecting network attacks within IoT environments.

5.3.1 Application of COD. For the Botnet-IoT and Ton-IoT datasets, the application of the COD method is relatively straightforward. Initially, all numerical features were taken into account for both datasets. Subsequently, a selection of these features was extracted, with certain features being excluded as detailed in Table 3 for Botnet-IoT features and in Table 4 for Ton-IoT features. The features extracted are the features in common within the datasets found in the literature and the attributes available within network traffics. Time-related features (like stime and ltime) were discarded to avoid biased outcomes, as well as sequence features (like seq and pkseqId). Other features not considered in Botnet-IoT are those specifically related to the Botnet-IoT dataset itself. These features, as cited in [31], are incorporated during the preprocessing phase and are not typically present in traffic data. Additionally, to maximize the model's generalizability, only attributes common to multiple datasets, or found in network traffic, were included, the result is new datasets with same number of logs but of specific features represented by COD_{ton} and COD_{bot} for Ton-IoT and Botnet-IoT datasets respectively.

5.3.2 Application of CTS. For the Botnet-IoT and Ton-IoT datasets, *CTS* method is an extension of the *COD* approach. In *CTS*, a time interval of $t = 1s$ applied to both COD_{ton} and COD_{bot} . Within this time frame, the features are aggregated as specified in Table 2. This process transforms the datasets into timeseries formats, referred

¹<https://github.com/lre-security-systems-team/jaber.2024.ares.git>

Table 3: Extracted and eliminated features of Bot-IoT dataset

Features Extracted	Features Eliminated
flgs_number	pkSeqID
dur	seq
proto_number	stime
pkts	ltime
bytes	TnBPSrcIP
state_number	TnBPDstIP
mean	TnP_PerProto
stddev	TnP_Per_Dport
sum	AR_P_Proto_P_SrcIP
min	AR_P_Proto_P_DstIP
max	N_IN_Conn_P_DstIP
spkts	N_IN_Conn_P_SrcIP
dpkts	AR_P_Proto_P_Sport
sbytes	AR_P_Proto_P_Dport
dbytes	Pkts_P_State_P_Protocol_P_DstIP
rate	Pkts_P_State_P_Protocol_P_SrcIP
srate	
drate	

Table 4: Extracted and eliminated features of Ton-IoT dataset

Features Extracted	Features Eliminated
duration	ts
src_bytes	ltime
dst_bytes	
src_pkts	
dst_pkts	
label	
type	

to as $CT_{S_{bot}}$ for Botnet-IoT and $CT_{S_{ton}}$ for Ton-IoT. Post transformation, there is typically a reduction in the number of data rows, which aids in further analysis and evaluation of the data.

5.3.3 Application of CTW. For the Botnet-IoT and Ton-IoT datasets, the CTW method is employed alongside the CTS approach. For CTW, different time-window ranges are set: $tw = 1min$ for $CT_{S_{bot}}$ with a minimum threshold of $MIN_{tw} \geq 10$, and $tw = 30sec$ for $CT_{S_{ton}}$ with $MIN_{tw} \geq 2$. During each tw , feature aggregation occurs as outlined in Table 2. This procedure converts $CT_{S_{bot}}$ and $CT_{S_{ton}}$ into a new dataset format represented by CTW_{bot} and CTW_{ton} respectively. In this format, each row in the dataset represents the aggregated data within the specified tw .

5.3.4 Application of SpectraTW. In the analysis of both Botnet-IoT and Ton-IoT datasets, employing *SpectraTW* necessitates additional processing for each time window T_i . *SpectraTW* itself is not bound by strict limitations, but constructing weighted edges within the network graph relies on selecting a specific feature. For the Ton-IoT datasets, we opted to use packet numbers as the criterion for edge weight across two different topologies within the time window: one topology is weighted by source packets, and the other by destination packets. This decision stems from the observation that packet counts are crucial for identifying most types of attacks.

In contrast, for the Botnet IoT dataset, we extract three topological graphs within the time window, each based on different edge weights: packets, bytes, and rate, respectively. In the context of TonIoT, two sets of eight metrics are extracted over two time windows: the first focuses on source packets, while the second targets destination packets. For each time window, metrics from μ_1 to μ_4 are calculated at two intervals, tw and $\frac{tw}{2}$. For the Botnet dataset, three sets of eight metrics each are computed from the three topologies. This process results in two datasets, DF_{bot} and DF_{ton} , with each row comprising vectors of spectral metrics alongside their corresponding labels.

5.4 Evaluations: Assessing Techniques and Results

In this section, we present a comprehensive evaluation of machine learning metrics, including the F1-Score, Balanced Accuracy, and Matthews Correlation Coefficient (MCC), for XGBoost classifier. We will assess the performance of XGBoost across various approaches, namely COD, CTS, CTW, and *SpectraTW*. Our discussion will delve into the analysis of these scores, offering insights into the effectiveness and efficiency of XGBoost within these methodologies.

5.4.1 Assessment of the Approach’s Effectiveness on the Datasets. Analyzing the performance of the XGBoost classifier across the five approaches using the Botnet-IoT dataset, we observe notable variances in the metrics. The COD approach presents a perfect F1-Score, Balanced Accuracy, and MCC of 1.0 for DDoS, indicating exemplary classification effectiveness. With CTS, we notice a slight decrease in the F1-Score and MCC for ScanService, suggesting a marginal reduction in precision and reliability of the model. In contrast, CTW enhances the model’s performance, achieving an MCC of 0.9683 for ScanService. The *SpectraTW* approach maintains this high standard with an F1-Score of 0.9994 for ScanService, underscoring its precision and recall. However, it is the *SpectraTW* that stands out, especially for the OS Fingerprint and Keylogging categories, yielding an F1-Score and MCC of 0.9953 and 0.9952 respectively for OS Fingerprint, which represents a substantial improvement over the baseline COD scores, indicating that the feature selection and combination strategies employed by *SpectraTW* significantly enhance the classifier’s ability to accurately identify and correlate predictions in these more challenging categories.

For the XGBoost classifier analyzing the Ton-IoT dataset, we discern distinct performance patterns across the approaches. The COD approach yields strong results with an F1-Score of 0.9764 for DDoS, yet this figure slightly diminishes in the CTS approach with a lower F1-Score of 0.6175 for DoS. The CTW approach does not markedly improve the metrics, but the *SpectraTW* approach elevates the performance significantly, particularly for Ransomware, achieving an F1-Score of 0.9938. The most pronounced improvements are observed with the *SpectraTW* approach, where the metrics for Ransomware exhibit high scores, reflected in an F1-Score of 1.0 and an MCC of 0.9932. This indicates that the *SpectraTW*, with its refined feature selection, enhances the XGBoost classifier’s ability to accurately predict and strongly correlate with the actual classifications, especially in complex attack scenarios such as Ransomware.

Table 5: Metrics evaluation over Botnet and TonIoT datasets

Botnet dataset		COD	CTS	CTW	SM
DDoS	F1 Score	1	0.8750	1	1
	Balanced Acc	1	0.8888	1	1
	MCC	1	0.8816	1	1
	Precision	1	1	1	1
	Recall	1	0.7777	1	1
ScanService	F1 Score	0.8937	0.9966	0.9990	0.9994
	Balanced Acc	0.9760	0.9133	0.9834	0.9942
	MCC	0.8835	0.8965	0.9797	0.9885
	Precision	0.8225	0.9939	0.9984	0.9994
	Recall	0.9783	0.9993	0.9997	0.9994
OS Fingerprint	F1 Score	0.2617	0.8235	0.9953	0.9953
	Balanced Acc	0.5804	0.8798	0.9953	0.9953
	MCC	0.3198	0.8240	0.9952	0.9952
	Precision	0.6594	0.8974	1	1
	Recall	0.1633	0.7608	0.9907	0.9907
Keylogging	F1 Score	0.5333	0.6666	1	1
	Balanced Acc	0.7856	0.7998	1	1
	MCC	0.5344	0.6703	1	1
	Precision	0.5	0.75	1	1
	Recall	0.5714	0.6	1	1
Prediction Time (sec)		0.24	0.01	0.03	0.03

TonIoT dataset		COD	CTS	CTW	SM
DDoS	F1 Score	0.9764	0.6175	0.9943	1
	Balanced Acc	0.9833	0.7628	0.9971	1
	MCC	0.9752	0.6256	0.9943	1
	Precision	0.9857	0.7465	0.9943	1
	Recall	0.9674	0.5265	0.9943	1
DoS	F1 Score	0.9837	0.9054	0.9957	1
	Balanced Acc	0.9902	0.9403	0.9964	1
	MCC	1	0.8816	1	1
	Precision	0.9863	0.9304	0.9985	1
	Recall	0.9811	0.8817	0.9928	1
Scanning	F1 Score	0.9890	0.3363	0.9938	1
	Balanced Acc	0.9959	0.6102	0.9995	1
	MCC	0.9884	0.3631	0.9933	1
	Precision	0.9852	0.6411	0.9877	1
	Recall	0.9928	0.2279	1	1
Ransomware	F1 Score	0.8290	0.2978	0.9243	0.9949
	Balanced Acc	0.9131	0.5973	0.9526	0.9949
	MCC	0.8193	0.3450	0.9240	0.9948
	Precision	0.8218	0.6202	0.9438	1
	Recall	0.8364	0.196	0.9057	0.9898
SQL Injection	F1 Score	0.9735	0.8445	0.8428	0.9972
	Balanced Acc	0.9808	0.9040	0.8732	0.9999
	MCC	0.9721	0.8433	0.8480	0.9972
	Precision	0.9848	0.8827	0.9762	0.9946
	Recall	0.9624	0.8094	0.7468	1
Password	F1 Score	0.9808	0.7304	0.9148	0.9939
	Balanced Acc	0.9882	0.9522	0.9748	0.9966
	MCC	0.9798	0.7363	0.9125	0.9937
	Precision	0.9844	0.6022	0.8786	0.9943
	Recall	0.9772	0.9280	0.9541	0.9935
XSS	F1 Score	0.8710	0.6731	1	1
	Balanced Acc	0.9509	0.8043	1	1
	MCC	0.8645	0.6753	1	1
	Precision	0.8336	0.7513	1	1
	Recall	0.9120	0.6096	1	1
Backdoor	F1 Score	0.9985	0.8589	0.9928	0.9995
	Balanced Acc	0.9989	0.8928	0.9967	0.9995
	MCC	0.9984	0.8563	0.9923	0.9995
	Precision	0.999	0.9437	0.9916	1
	Recall	0.9980	0.7880	0.9939	0.999
MitM	F1 Score	0.7239	0.4542	0.7640	1
	Balanced Acc	0.8733	0.6747	0.8148	1
	MCC	0.7235	0.4742	0.7818	1
	Precision	0.7017	0.6458	0.9714	1
	Recall	0.7476	0.3502	0.6296	1
Prediction Time (sec)		0.83	0.31	0.21	0.15

5.5 Unveiling Key Features for Attack Detection Explainability

This section demonstrates the crucial role of spectral metrics in enhancing the performance of the XGBoost classifier, thereby increasing the accuracy in detecting attacks. As depicted in Fig. 7 and

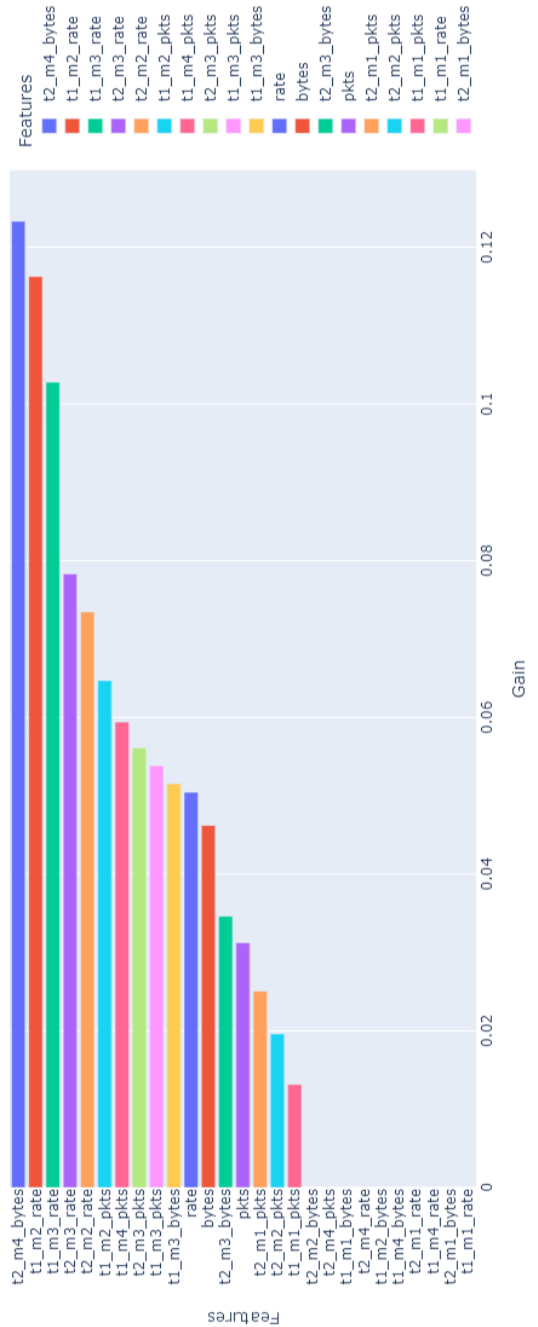
**Figure 6: Feature Importance of XGBoost in Botnet dataset over the SpectraTW approach Based on Gain.**

Fig. 7, the histogram showcases the most significant features based on their gain throughout the detection process using the XGBoost classifier. These figures highlight the significance of integrating spectral metrics with standard features when analyzing botnet and ton-IoT datasets. The 'Gain' metric, displayed on the x-axis, signifies each feature's contribution to the predictive capability of the model.

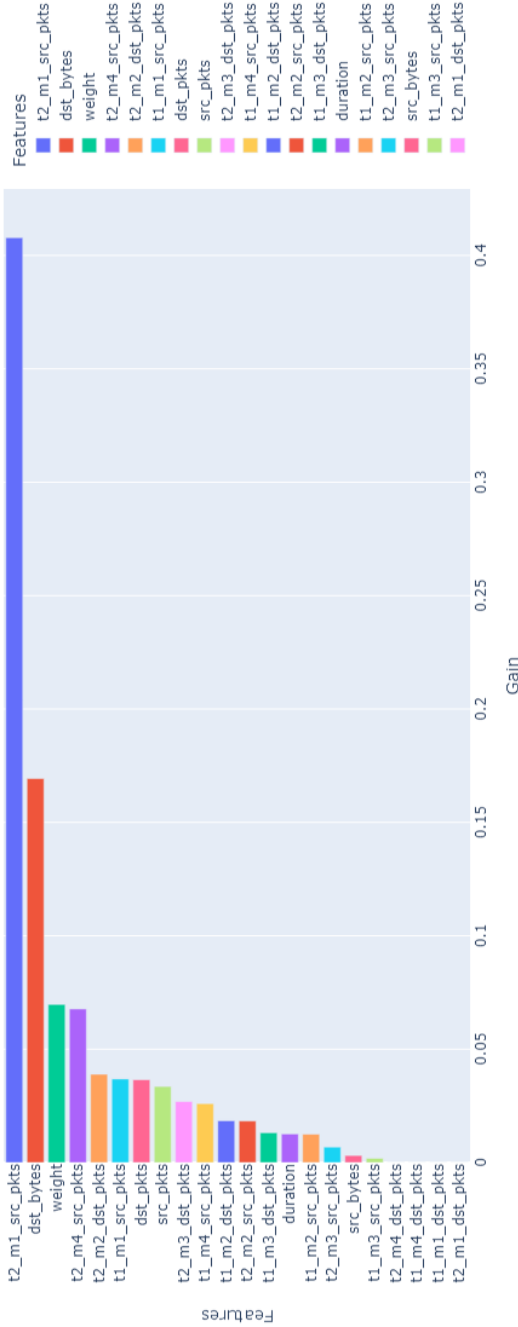


Figure 7: Feature Importance of XGBoost TonIoT dataset over the *SpectraTW* approach Based on Gain.

We introduce the notation $t_i_m_j_w$ corresponding to $SubTW_i$ with $i \in [1, 2]$, $j \in [1, 4]$ corresponds to the spectral metrics μ_j , and $w \in \{\text{"pkts"}, \text{"bytes"}, \text{"rate"}\}$ specifies the type of data quantified across the network edges.

The histogram, depicted in Fig. 6, illustrates the features derived from the botnet dataset. It reveals that $t2_m4_bytes$ and $t1_m2_rate$

are the features with the highest gains, showcasing a profound influence on the model's decision-making process. Conversely, simpler features such as bytes and rate display lesser gain values, indicating a relatively minor contribution.

The histogram, presented in Fig. 7, features data from the ton-IoT dataset, highlighting $t2_m1_src_pkts$ and dst_bytes as the primary influencers. It illustrates a broader diversity in gain values across features compared to both datasets, indicating a more evenly distributed impact among the features.

This comparative analysis showcase the flexibility of spectral metrics in capitalizing on the most indicative signals for precise predictions. The histograms show that spectral metrics from μ_1 to μ_4 hold consistently higher importance over specific features like bytes and packets across both datasets. This observation implies that these elements of network traffic are vital for anomaly detection concerning botnet security threats.

6 ATTACKS BEHAVIORS

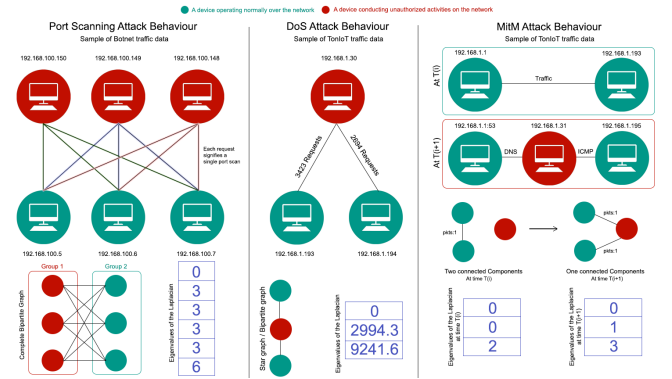


Figure 8: Behaviours of the three attacks: Port Scanning, DoS, and MitM taken from the simulated Botnet and TonIoT datasets with their corresponding spectrum.

In the datasets results shown in Table 5, attack distributions fall into the categories of exploitation, disruption, scanning, theft, and reconnaissance. This section provides a description of the attacks behaviours present and explains the role of spectral analysis in identifying these attacks.

A Denial of Service (DoS) attack disrupts system or network access by flooding it with traffic from a single source. In contrast, a Distributed Denial of Service (DDoS) attack increase this disruption by coordinating traffic from multiple compromised devices, magnifying its impact and severity. Fig. 8 provides a clear illustration of TonIoT traffic data during a DoS attack within a time-window snapshot. During such attacks, spectral metrics, particularly *Connectedness* and *Wiriness*, remain adaptable despite the number of requests shown by edge weights overwhelming the target node. These metrics, reliant on algebraic connectivity and the graph's largest degrees, are notably displayed in Fig. 8, showcasing their influence on eigenvalues, which experience substantial increases. Consequently, there are noticeable alterations in connections between the attacking sources and the target device. Figure 15 illustrates the behavior of spectral metrics during the DDoS attack.

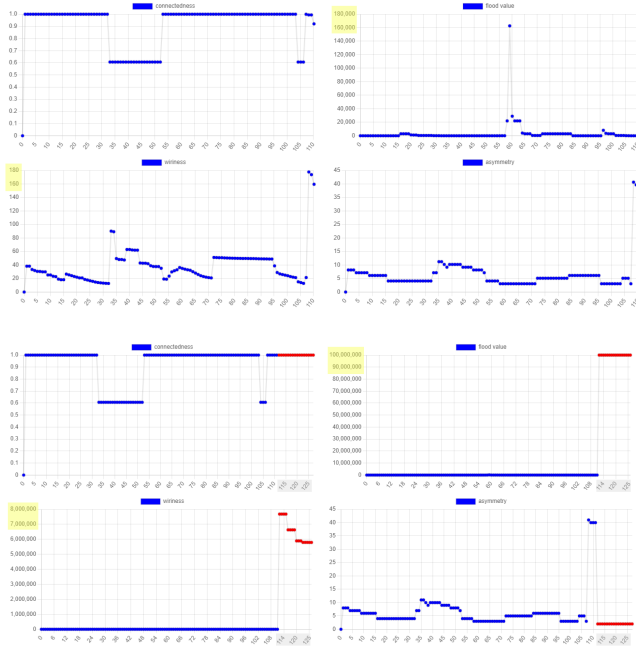


Figure 9: Performance of spectral metrics at the end of each time window in the Botnet dataset. The upper four curves illustrate the changes in spectral metrics from T_{w_0} to $T_{w_{10}}$ during normal traffic; the lower four curves depict the behavior of spectral metrics throughout the attack from $T_{w_{10}}$ to $T_{w_{125}}$, highlighted in red to indicate the presence of a DDoS attack.

The first four curves represent the behavior of the *Connectedness*, *Flooding*, *Wiriness*, and *Asymmetry* spectral metrics over the time-windows up to $T_{w_{10}}$, clearly showing how the metrics propagate at the end of each time-window and indicating the propagation pattern. Beyond $T_{w_{10}}$, up to $T_{w_{125}}$, where the DDoS attack occurs, there is a rapid increase in the behavior of *Flooding* and *Wiriness*, reaching above ninety thousand million and seven million, respectively. Additionally, after this significant increase in *Flooding* and *Wiriness*, both curves exhibit a flat pattern, which is explained by the change in the scale of the graphs during the DDoS attack.

In a Man-in-the-Middle (MitM) attack, an attacker clandestinely intercepts and may alter the messages between two parties who believe they are communicating directly. This enables the attacker to eavesdrop on or manipulate the information being exchanged. Fig. 8 illustrates the behavior of a MitM attack in a time-window snapshot from the TonIoT dataset. Initially at time T_i , two endpoints engage in normal communication and packet exchange. However, at time T_{i+1} , a new anonymous device intervenes and initiates a DNS spoofing attack with one of the parties over port 53. Additionally, the attacker employs the ICMP protocol for reconnaissance activities with the other party. The spectral metric *Connectedness* plays a crucial role in detecting any unauthorized device attempting to bridge two subnetworks. This detection is evident through variations in the graph’s connected components count over time, as well as any modifications in the content of packet requests.

Port scanning as shown in Fig. 8 is utilized in scanning attacks to discover open ports on a network of different devices, while OS fingerprinting aims to collect information about the operating system and its protocols. These attacks are characterized by continuous port scanning, evident through an uptick in attempted connections and a rise in the volume and frequency of data packets directed at the victim. *SpectraTW* utilizes spectral metric features that rely on communication data, influencing its spectral properties, such as *Flooding*, which signifies the inter-connectivity between nodes. These metrics also take into account factors like the count, rate, and size of packets per session across time periods. Moreover, Fig. 8 displays a snapshot from the Botnet IoT dataset, depicting the end of a time-window during which port scanning occurred. In this snapshot, a complete bipartite graph is clearly visible, illustrating the topology of the network. This highlights the importance of spectral metrics in analyzing the topological patterns in dynamic graphs and quantifying these patterns within the spectral metrics. Scanning attacks focus on various sub-networks, impacting its spectral features and consequently modifying the network graph’s structure. Following the initial scanning, attackers often move to tactics such as keylogging of sensitive data, typically leading to an increase in network activity and often targeting specific ports like port 22 for SSH. This stage is marked by grouped data exchanges interrupted by short breaks, signaling the end of each data transfer. *SpectraTW* adeptly manages multiple time windows, reflecting different conditions and ports within the same period. It assesses several graphs generated from data concerning packets, rates, and bytes, crucial for identifying theft attacks.

7 CONCLUSIONS AND PERSPECTIVES

This study demonstrates the promising potential of utilizing the Laplacian spectrum of a graph to detect various cyber attacks effectively. The Laplacian spectrum provides comprehensive insights into network topology, enabling the detection of threats in a dynamic and easily interpretable manner through the proposed metrics. As part of future research directions, we aim to develop metrics tailored to other recurring patterns (like k-partite graphs) observed in cybersecurity traffic datasets, thereby enhancing the comprehensiveness of our study. Additionally, we intend to explore the application of our approach to directed graphs, considering the significance of traffic direction in identifying the sources of requests.

The proposed approach underscores the efficacy of spectral analysis in identifying cyber threats within medical IoT networks. Moreover, it highlights the successful application of XGBoost in anomaly detection, with *SpectraTW* demonstrating notable efficiency across various attack scenarios. Notably, for existing datasets on medical IoT attacks, *SpectraTW* achieves a significant improvement of detection for certain threats like MitM attacks and demonstrates its effectiveness wrt. state-of-the-art approaches for detecting more complex attacks such as Ransomware.

ACKNOWLEDGMENTS

The authors acknowledge the support of the Région Grand-Est and École Pour l’Informatique et les Techniques Avancées (EPITA) for the joint funding of the XDGMed Project. Their contributions have been invaluable to the research and development efforts. The code

supporting this work is available at <https://github.com/lre-security-systems-team/jaber.2024.ares.git>.

REFERENCES

- [1] Niall M Adams and Nicholas A Heard. 2016. *Dynamic networks and cyber-security*. Vol. 1. World Scientific.
- [2] Mohiuddin Ahmed, Abdun Naser Mahmood, and Md Rafiqul Islam. 2016. A survey of anomaly detection techniques in financial domain. *Future Generation Computer Systems* 55 (2016), 278–288.
- [3] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. 2020. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *Ieee Access* 8 (2020), 165130–165150.
- [4] Gianmarco Baldini. 2022. Detection of cybersecurity spoofing attacks in vehicular networks with recurrence quantification analysis. *Computer Communications* 191 (2022), 486–499.
- [5] Frank Bauer and Jürgen Jost. 2009. Bipartite and neighborhood graphs and the spectrum of the normalized graph Laplacian. *arXiv preprint arXiv:0910.3118* (2009).
- [6] Dhruba Kumar Bhattacharyya and Jugal Kumar Kalita. 2019. *Network anomaly detection: A machine learning perspective*. Chapman and Hall/CRC.
- [7] Cemal Gagatay Bilgin and Bülent Yener. 2006. Dynamic network evolution: Models, clustering, anomaly detection. *IEEE Networks* 1 (2006).
- [8] Giuseppe Bonaccorso. 2017. *Machine learning algorithms*. Packt Publishing Ltd.
- [9] Tim M Booi, Irina Chiscop, Erik Meeuwissen, Nour Moustafa, and Frank TH Den Hartog. 2021. ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets. *IEEE Internet of Things Journal* 9, 1 (2021), 485–496.
- [10] Benjamin Bowman and H Howie Huang. 2021. Towards Next-Generation Cybersecurity with Graph AI. *ACM SIGOPS Operating Systems Review* 55, 1 (2021), 61–67.
- [11] Benjamin Bowman, Craig Laprade, Yuede Ji, and H Howie Huang. 2020. Detecting Lateral Movement in Enterprise Computer Networks with Unsupervised Graph {AI}. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)* 2020. 257–268.
- [12] Steve Butler, Fan Chung, et al. 2006. Spectral graph theory. *Handbook of linear algebra* (2006), 47.
- [13] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural temporal graph neural networks for anomaly detection in dynamic graphs. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3747–3756.
- [14] Pin-Yu Chen, Sutanay Choudhury, and Alfred O Hero. 2016. Multi-centrality graph spectral decompositions and their application to cyber intrusion detection. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 4553–4557.
- [15] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [16] Yu Chen and Kai Hwang. 2006. Collaborative detection and filtering of shrew DDoS attacks using spectral analysis. *J. Parallel and Distrib. Comput.* 66, 9 (2006), 1137–1151.
- [17] Chen-Mou Cheng, HT Kung, and Koan-Sin Tan. 2002. Use of spectral analysis in defense against DoS attacks. In *Global Telecommunications Conference, 2002. GLOBECOM'02. IEEE*, Vol. 3. IEEE, 2143–2148.
- [18] Fan RK Chung. 1997. *Spectral graph theory*. Vol. 92. American Mathematical Soc.
- [19] Rishabh Das and Thomas H Morris. 2017. Machine learning and cyber security. In *2017 international conference on computer, electrical & communication engineering (ICCECE)*. IEEE, 1–7.
- [20] Nair Maria Maia De Abreu. 2007. Old and new results on algebraic connectivity of graphs. *Linear algebra and its applications* 423, 1 (2007), 53–73.
- [21] Siemon de Lange, Marcel de Reus, and Martijn Van Den Heuvel. 2014. The Laplacian spectrum of neural networks. *Frontiers in computational neuroscience* 7 (2014), 189.
- [22] Dimitar V Dimitrov. 2016. Medical internet of things and big data in healthcare. *Healthcare informatics research* 22, 3 (2016), 156–163.
- [23] M Umar Farooq, Muhammad Waseem, Sadia Mazhar, Anjum Khairi, and Talha Kamal. 2015. A review on internet of things (IoT). *International journal of computer applications* 113, 1 (2015), 1–7.
- [24] Behrouz A Forouzan. 2007. *Data communications and networking*. Huga Media.
- [25] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
- [26] Mostafa Haghi, Kerstin Thurov, and Regina Stoll. 2017. Wearable devices in medical internet of things: scientific research and commercially available devices. *Healthcare informatics research* 23, 1 (2017), 4–15.
- [27] Ayush Hariharan, Ankith Gupta, and Trisha Pal. 2020. Camlpad: Cybersecurity autonomous machine learning platform for anomaly detection. In *Future of Information and Communication Conference*. Springer, 705–720.
- [28] Nick Heard and Patrick Rubin-Delanchy. 2016. Network-wide anomaly detection via the Dirichlet process. In *2016 IEEE Conference on Intelligence and Security Informatics (ISI)*. IEEE, 220–224.
- [29] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [30] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Jill Slay. 2018. Towards developing network forensic mechanism for botnet activities in the IoT based on machine learning techniques. In *Mobile Networks and Management: 9th International Conference, MONAMI 2017, Melbourne, Australia, December 13-15, 2017, Proceedings 9*. Springer, 30–44.
- [31] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. 2019. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Generation Computer Systems* 100 (2019), 779–796.
- [32] Michael Krivelevich and Benny Sudakov. 2003. The largest eigenvalue of sparse random graphs. *Combinatorics, Probability and Computing* 12, 1 (2003), 61–72.
- [33] Yongsheng Liu, Wenyu Chen, Hong Qu, SM Hasan Mahmud, and Kebin Miao. 2021. Weakly supervised image classification and pointwise localization with graph convolutional networks. *Pattern Recognition* 109 (2021), 107596.
- [34] Ricardo A López. 2007. Computer networks. A top-down approach featuring Internet. *Journal of Computer Science and Technology* 7, 2 (2007), 194.
- [35] Somayya Madakam, Vihar Lake, Vihar Lake, Vihar Lake, et al. 2015. Internet of Things (IoT): A literature review. *Journal of Computer and Communications* 3, 05 (2015), 164.
- [36] Batta Mahesh. 2020. Machine learning algorithms-a review. *International Journal of Science and Research (IJSR)*, [Internet] 9, 1 (2020), 381–386.
- [37] Russell Merris. 1994. Laplacian matrices of graphs: a survey. *Linear algebra and its applications* 197 (1994), 143–176.
- [38] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: an ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089* (2018).
- [39] Nour Moustafa. 2021. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society* 72 (2021), 102994.
- [40] Nour Moustafa, Mohiuddin Ahmed, and Sherif Ahmed. 2020. Data analytics-enabled intrusion detection: Evaluations of ToN_IoT linux datasets. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 727–735.
- [41] Nour Moustafa, Marwa Keshky, Essam Debiez, and Helge Janicke. 2020. Federated TON_IoT Windows datasets for evaluating AI-based security applications. In *2020 IEEE 19th international conference on trust, security and privacy in computing and communications (TrustCom)*. IEEE, 848–855.
- [42] Jordan Noble and Niall Adams. 2018. Real-time dynamic network anomaly detection. *IEEE Intelligent Systems* 33, 2 (2018), 5–18.
- [43] Sergio Iglesias Pérez, Santiago Moral-Rubio, and Regino Criado. 2021. A new approach to combine multiplex networks and time series attributes: Building intrusion detection systems (IDS) in cybersecurity. *Chaos, Solitons & Fractals* 150 (2021), 111143.
- [44] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
- [45] Iqbal H Sarker. 2021. Machine learning: Algorithms, real-world applications and research directions. *SN computer science* 2, 3 (2021), 160.
- [46] Kamran Shaukat, Suhuai Luo, Vijay Varadharajan, Ibrahim A Hameed, Shan Chen, Dongxi Liu, and Jiaming Li. 2020. Performance comparison and current challenges of using machine learning techniques in cybersecurity. *Energies* 13, 10 (2020), 2509.
- [47] Amanpreet Singh, Narina Thakur, and Aakanksha Sharma. 2016. A review of supervised machine learning algorithms. In *2016 3rd international conference on computing for sustainable global development (INDIACom)*. Ieee, 1310–1315.
- [48] William Stallings. 2003. *Network security essentials: Applications and standards, 4/e*. Pearson Education India.
- [49] Andrew Tomlinson, Jeremy Bryans, Siraj Ahmed Shaikh, and Harsha Kumara Kaluturage. 2018. Detection of automotive CAN cyber-attacks by identifying packet timing anomalies in time windows. In *2018 48th Annual IEEE/IFIP international conference on dependable systems and networks workshops (DSN-W)*. IEEE, 231–238.
- [50] M Uma and Ganapathi Padmavathi. 2013. A Survey on Various Cyber Attacks and their Classification. *Int. J. Netw. Secur.* 15, 5 (2013), 390–396.
- [51] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [52] Jie Wan, Munassar AAH Al-awlaqi, MingSong Li, Michael O'Grady, Xiang Gu, Jin Wang, and Ning Cao. 2018. Wearable IoT enabled real-time health monitoring system. *EURASIP Journal on Wireless Communications and Networking* 2018, 1 (2018), 1–10.
- [53] Wei Wang and Thomas E Daniels. 2006. Diffusion and graph spectral methods for network forensic analysis. In *Proceedings of the 2006 workshop on New security paradigms*. 99–106.

- [54] Naganand Yadati, Madhav Nimishakavi, Prateek Yadav, Anand Louis, and Partha Talukdar. 2018. Hypergc: Hypergraph convolutional networks for semi-supervised classification. *arXiv preprint arXiv:1809.02589* 22 (2018).
- [55] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, and Xinzhen He. 2017. A deep learning approach for intrusion detection using recurrent neural networks. *Ieee Access* 5 (2017), 21954–21961.
- [56] Xiaowei Ying, Xintao Wu, and Daniel Barbara. 2010. Spectrum based fraud detection in social networks. In *Proceedings of the 17th ACM conference on Computer and communications security*. 747–749.
- [57] Aimei Yu, Mei Lu, and Feng Tian. 2004. On the spectral radius of graphs. *Linear algebra and its applications* 387 (2004), 41–49.